ADOPT-03: Success Metrics

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 3 of 5 |
> **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

What gets measured gets improved. This notebook defines the key success metrics for an observability practice: Mean Time to Detect (MTTD), Mean Time to Resolve (MTTR), change failure rate, problem count trends, and alert noise ratio. For each metric, we provide a baseline DQL query, explain how to interpret results, and describe how to track improvement over time. These metrics translate observability investment into language that leadership understands.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`storage:events:read`, `storage:logs:read`, `storage:metrics:read`
Data	At least 7 days of detected problem data for meaningful baselines
Audience	SREs, engineering managers, VP of Engineering, CTO

1. Why Success Metrics Matter

Observability platforms generate enormous volumes of data. Without defined success metrics, it is impossible to answer the fundamental question: **"Is our observability practice making us better?"**

Success metrics serve three purposes:

Purpose	Audience	Example
Operational improvement	SRE / Platform teams	MTTR reduced from 45 min to 12 min
Business justification	Leadership / Finance	60% fewer customer-impacting incidents
Continuous improvement	All teams	Alert noise ratio decreased from 65% to 15%

The DORA (DevOps Research and Assessment) metrics framework provides industry-standard benchmarks. We map Dynatrace capabilities to DORA metrics throughout this notebook.

2. Mean Time to Detect (MTTD)

MTTD measures how long it takes from when a problem begins to when it is detected. In Dynatrace, Dynatrace Intelligence continuously analyzes telemetry and opens problems automatically. MTTD is the difference between problem start time and the timestamp when Dynatrace Intelligence created the problem event.

Why It Matters

- Lower MTTD means faster awareness of issues
- Dynatrace Intelligence typically detects anomalies within minutes — human detection often takes hours
- Tracking MTTD validates that your instrumentation and alerting are working

2.1 MTTD Over the Last 7 Days

Dynatrace Intelligence problems have an `event.start` timestamp (when the underlying condition began). By comparing this against the record timestamp, we can estimate detection lag.

```
// Estimate MTTD: gap between problem start and Dynatrace Intelligence
detection
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| fieldsAdd detection_lag_minutes = (toDouble(timestamp - event.start)) /
60000000000.0
| summarize
    avg_mttd_minutes = avg(detection_lag_minutes),
    median_mttd_minutes = median(detection_lag_minutes),
    p95_mttd_minutes = percentile(detection_lag_minutes, 95),
    problem_count = count()
```

> **Interpreting MTTD Results:**
> - **< 5 minutes** — Excellent. Dynatrace Intelligence is detecting issues rapidly.
> - **5-15 minutes** — Good. Typical for most environments.
> - **> 15 minutes** — Review alert configurations and instrumentation coverage.

3. Mean Time to Resolve (MTTR)

MTTR measures the total time from problem detection to resolution. It captures the full incident lifecycle: detection, triage, investigation, remediation, and verification.

Why It Matters

- MTTR is the single most important operational metric for reliability
- DORA classifies teams as "Elite" when MTTR is under 1 hour
- Dynatrace can reduce MTTR by providing root cause analysis automatically

3.1 MTTR Trend Over 7 Days

```
// MTTR trend: average resolution time in hours, trended by day
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| filter maintenance.is_under_maintenance == false
| makeTimeseries avg_mttr_hours = avg(toLong(resolved_problem_duration) /
3600000000000.0), time:event.end
```

3.2 MTTR Summary Statistics

```
// MTTR summary: average, median, and p95 resolution time in hours
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| filter maintenance.is_under_maintenance == false
| fieldsAdd duration_hours = toLong(resolved_problem_duration) /
3600000000000.0
| summarize
    avg_mttr = avg(duration_hours),
    median_mttr = median(duration_hours),
    p95_mttr = percentile(duration_hours, 95),
    total_problems = count()
```

3.3 MTTR by Problem Category

Different problem categories often have very different resolution times. Breaking MTTR down by category helps identify which problem types need process improvement.

```
// MTTR breakdown by problem category
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| fieldsAdd duration_hours = toLong(resolved_problem_duration) /
3600000000000.0
| summarize
    avg_mttr = avg(duration_hours),
    problem_count = count(),
    by:{event.category}
| sort avg_mttr desc
```

> **DORA MTTR Benchmarks:**
>
> | Performance Level | MTTR |
> |---|---|
> | Elite | < 1 hour |
> | High | < 1 day |
> | Medium | < 1 week |
> | Low | > 1 week |

4. Problem Count Trends

Tracking the total number of problems over time reveals whether your environment is becoming more stable or more volatile. A decreasing trend indicates that root causes are being addressed, not just symptoms.

4.1 Weekly Problem Trend

```
// Weekly problem count trend over the last 30 days
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| fieldsAdd week = getWeekOfYear(timestamp)
| summarize problem_count = count(), by:{week}
| sort week asc
```

4.2 Problems by Category Over Time

```
// Problem trend by category over the last 7 days
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| makeTimeseries problem_count = count(), interval:1d, by:{event.category}
```

4.3 Top Recurring Problem Types

Identifying the most frequent problem types helps prioritize remediation efforts.

```
// Top 10 recurring problem types in the last 30 days
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| summarize occurrences = count(), by:{event.name}
| sort occurrences desc
| limit 10
```

5. Alert Quality — Noise Ratio

Alert fatigue is one of the greatest threats to operational effectiveness. When teams are overwhelmed by noisy, duplicate, or non-actionable alerts, they stop responding — and real problems get missed.

5.1 Alert Noise Analysis

```
// Alert noise analysis: actionable vs non-actionable problems
fetch dt.davis.problems, from:-7d
| summarize
    total = count(),
    actionable = countIf(dt.davis.is_frequent_event == false and
dt.davis.is_duplicate == false),
    frequent = countIf(dt.davis.is_frequent_event == true),
    duplicate = countIf(dt.davis.is_duplicate == true)
| fieldsAdd actionable_pct = round(toDouble(actionable) / toDouble(total) *
100, decimals: 1)
| fieldsAdd noise_pct = round(100 - toDouble(actionable_pct), decimals: 1)
```

5.2 Noise Trend Over Time

```
// Daily noise trend: percentage of non-actionable problems per day
fetch dt.davis.problems, from:-7d
| makeTimeseries
    total = count(),
    noisy = countIf(dt.davis.is_frequent_event == true or
dt.davis.is_duplicate == true),
    interval:1d
```

> **Target:** Keep the noise ratio below 20%. If it exceeds 50%, consider:
> - Adjusting Dynatrace Intelligence sensitivity settings
> - Configuring maintenance windows for known noisy periods
> - Reviewing alerting profiles and notification filters

6. Change Failure Rate

Change failure rate (CFR) measures the percentage of deployments that cause problems. It is a core DORA metric and directly reflects deployment quality and testing effectiveness.

Measuring CFR with Dynatrace

Dynatrace tracks deployment events and can correlate them with detected problems. The basic approach:

1. Count deployment events in a time window
2. Count problems that started within a defined window after a deployment
3. Calculate the ratio

6.1 Deployment Event Count

```
// Count deployment events in the last 7 days
fetch events, from:-7d
| filter event.kind == "CUSTOM_DEPLOYMENT"
| summarize deployment_count = count()
```

6.2 Problems Correlated with Deployments

When Dynatrace Intelligence detects a problem, it often identifies a recent deployment as the root cause. Problems tagged with deployment correlation indicate change-related failures.

```
// Problems in the last 7 days that may correlate with changes
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| summarize
    total_problems = count(),
    by:{event.category}
| sort total_problems desc
```

> **DORA Change Failure Rate Benchmarks:**
>
> | Performance Level | CFR |
> |---|---|
> | Elite | < 5% |
> | High | 5-10% |
> | Medium | 10-15% |

Performance Level	CFR
Low	> 15%

7. Establishing Baselines

A baseline is a snapshot of your current metrics that serves as the starting point for improvement tracking. Without baselines, you cannot measure progress.

Baseline Template

Record your current values and set improvement targets:

Metric	Current Baseline	3-Month Target	6-Month Target
MTTD	___ minutes	___ minutes	___ minutes
MTTR	___ hours	___ hours	___ hours
Weekly Problem Count	___ problems	___ problems	___ problems
Alert Noise Ratio	___%	___%	___%
Change Failure Rate	___%	___%	___%

Best Practices for Baseline Setting

- Use a **30-day window** for the initial baseline to smooth out anomalies
- Exclude maintenance windows and known outage periods
- Filter out frequent and duplicate events for cleaner metrics
- Record baselines per team or service if organizational structure allows
- **Re-baseline quarterly** to account for growth and environmental changes

8. Summary and Next Steps

Key Takeaways

- MTTD and MTTR are the two most important operational metrics — track them first
- Alert noise ratio directly impacts team effectiveness and should be monitored monthly
- Problem count trends reveal whether your environment is stabilizing or degrading
- DORA metrics (MTTR, CFR, deployment frequency) bridge observability and DevOps performance
- Baselines must be established before improvement can be measured

Next Steps

- Proceed to **ADOPT-04: Team Enablement** to build role-based learning paths for your organization
- Create a recurring notebook or dashboard that runs these queries weekly
- Share baseline metrics with leadership to establish accountability

---